



One Project One Week Hackathon 2026 by Technortal



System Architecture to Go-Live Production Deployment

[Aung Kyaw Minn](#)

Solution Architect @ AYA Innovation Labs



Table of Contents

- System Architecture Fundamentals
- Database & Performance Optimization
- Building Scalable APIs
- Security & Error Handling
- Production Deployment Practices

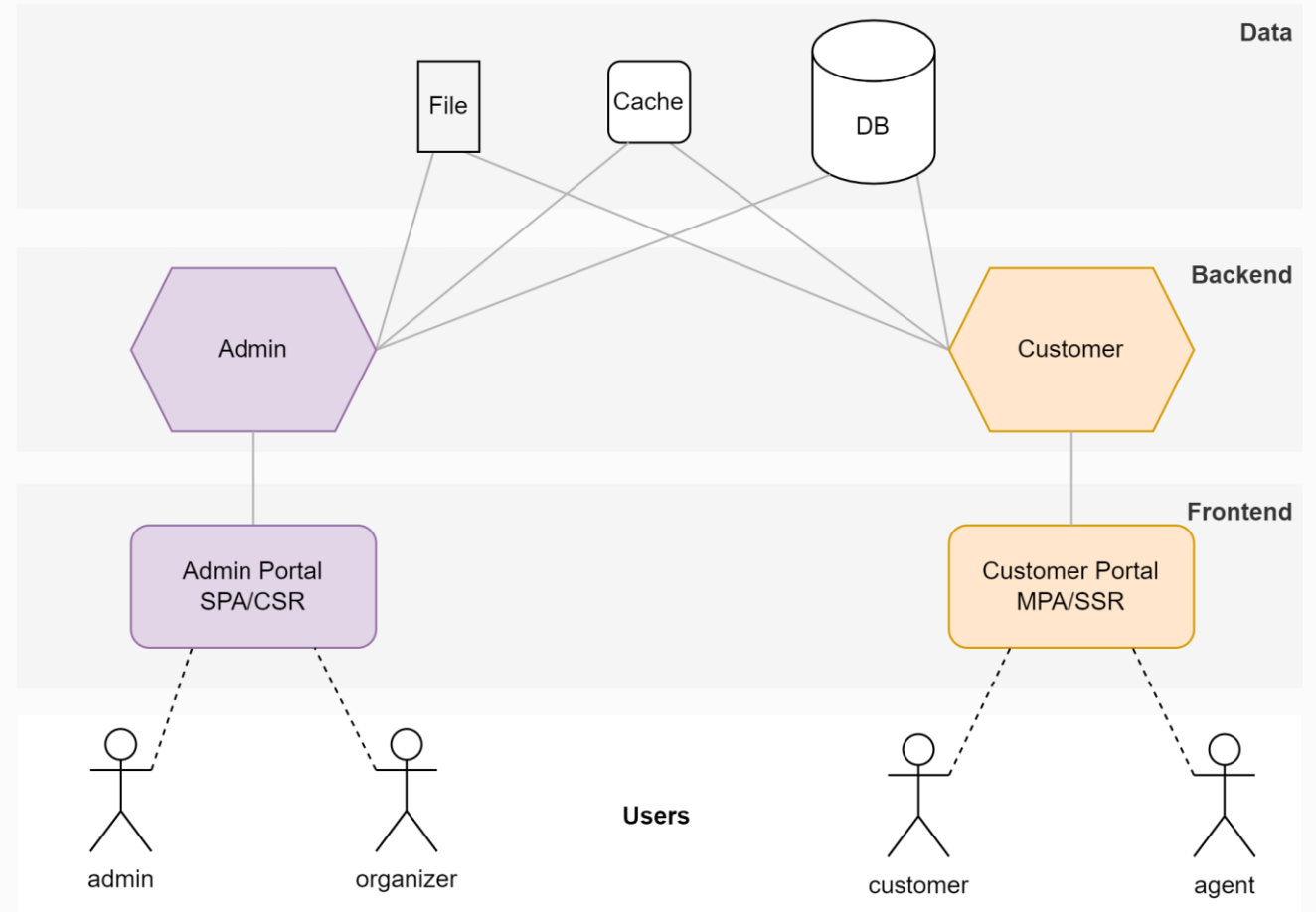
System Architecture Fundamentals

Blueprint, Components and Flows

System Architecture

Conceptual Design of the whole system

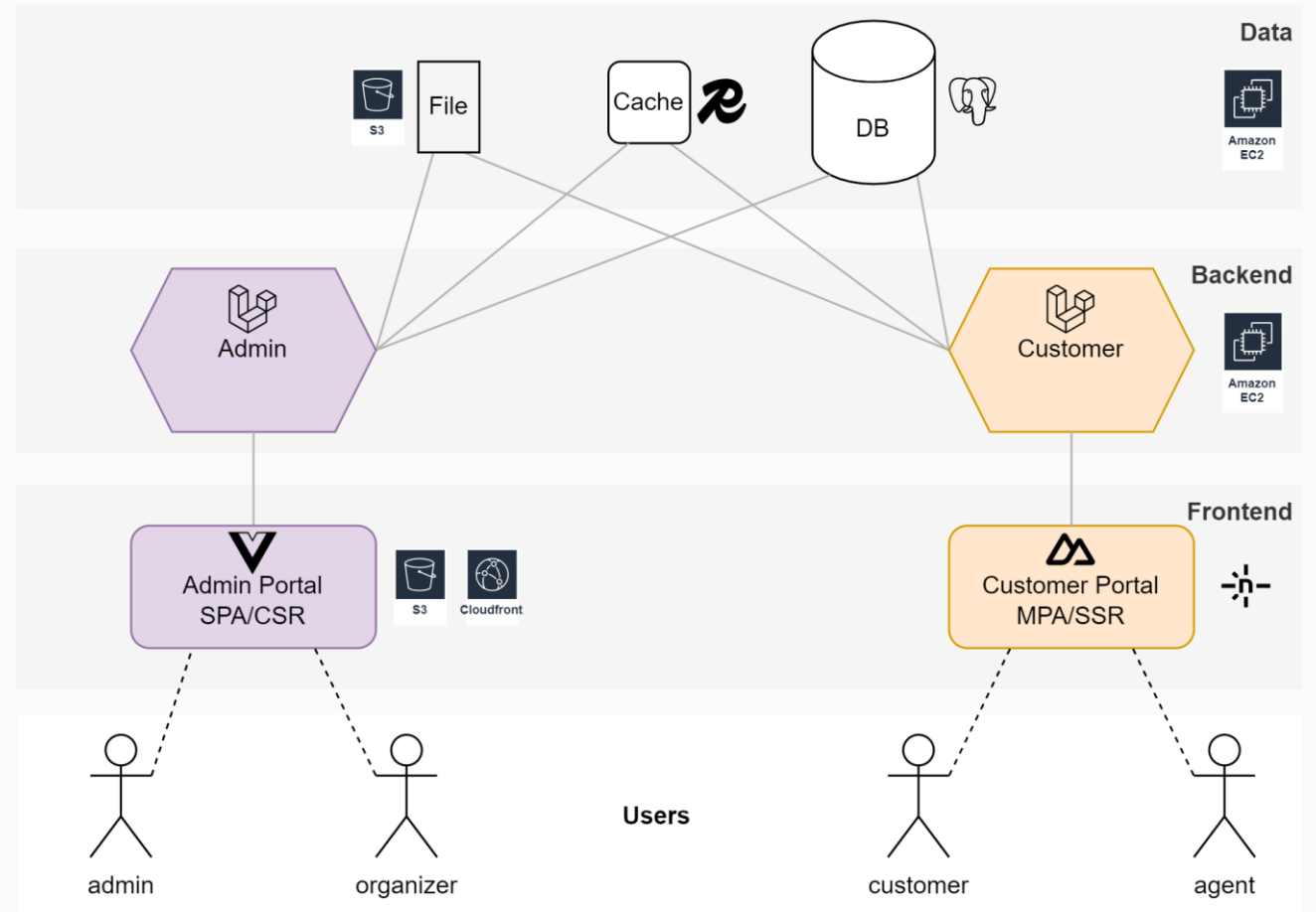
- Users
- Frontend: User Interfaces
 - Web App, Mobile App, Desktop App, ...
- Backend: Services
 - Core Application, Email Service, ...
- Data: Databases & Storage
 - RDS, NoSQL, Redis, AWS S3, ...



Architecture Decision

Fundamental design choice that dictates how a system will be built.

- Technology Stack
 - Backend Framework?
 - Frontend Framework?
- Database Systems & Storage
 - RDS? NoSQL? EBS? S3?
- Communication Protocols
 - REST? WebSocket? GraphQL? gRPC?
- Infrastructure & Hosting
 - Cloud? AWS? Google? DO? On-Prem?



System Designs

Tactical, detailed implementation of the broader system architecture.

- Data Model and Schema
 - Conceptual, Logical and Physical
- System Detail Diagram
 - Process Diagram, Flowchart
- *Algorithm or Pseudo Code*
- API Contracts

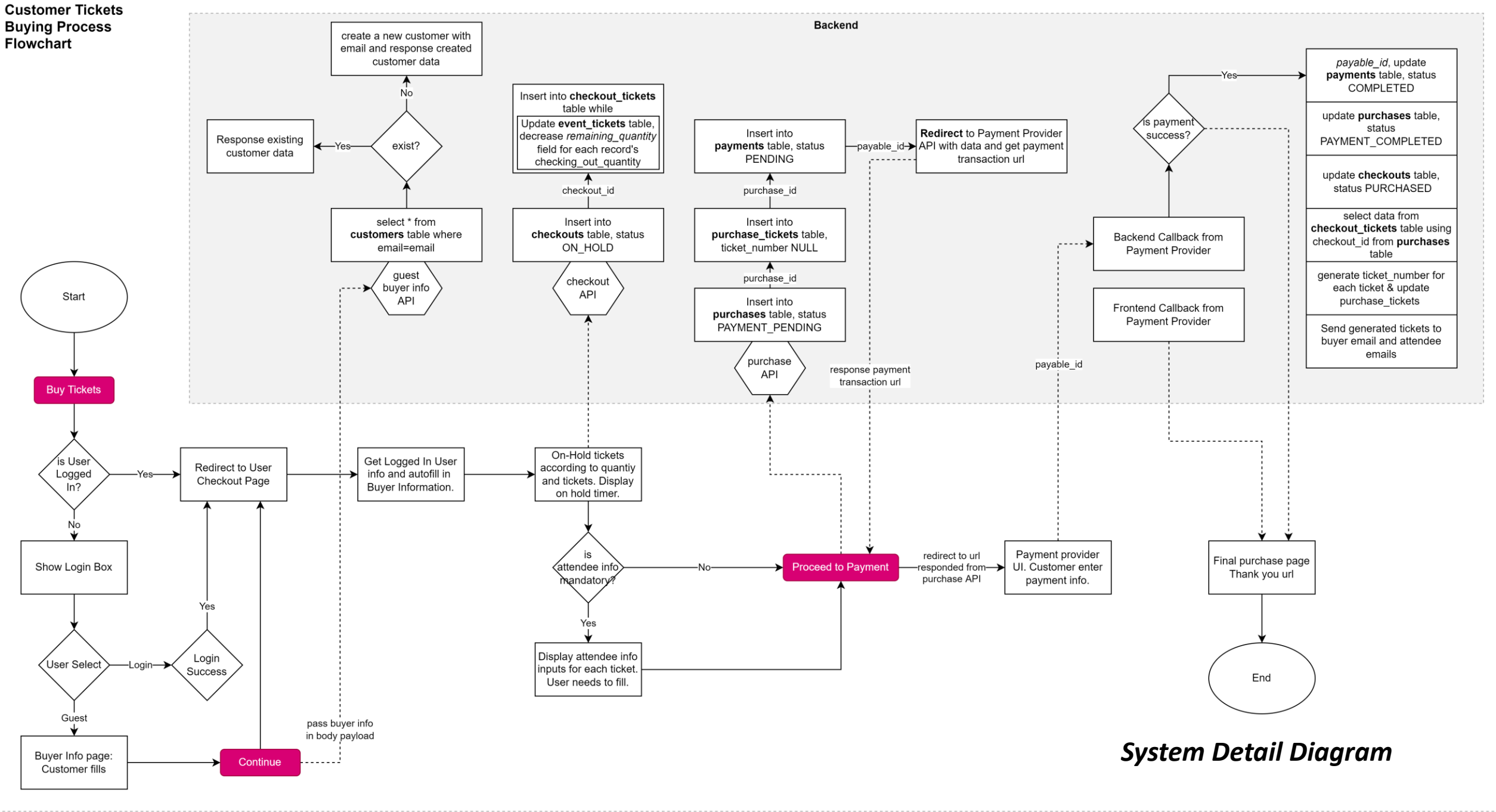


<https://dbdocs.io/ajminn207/Event-Ticketing-Platform>

<https://dbdiagram.io/d/eventickat-667f5c409939893dae9335aa>

Data Model & Schema

Customer Tickets
Buying Process
Flowchart



System Detail Diagram


```

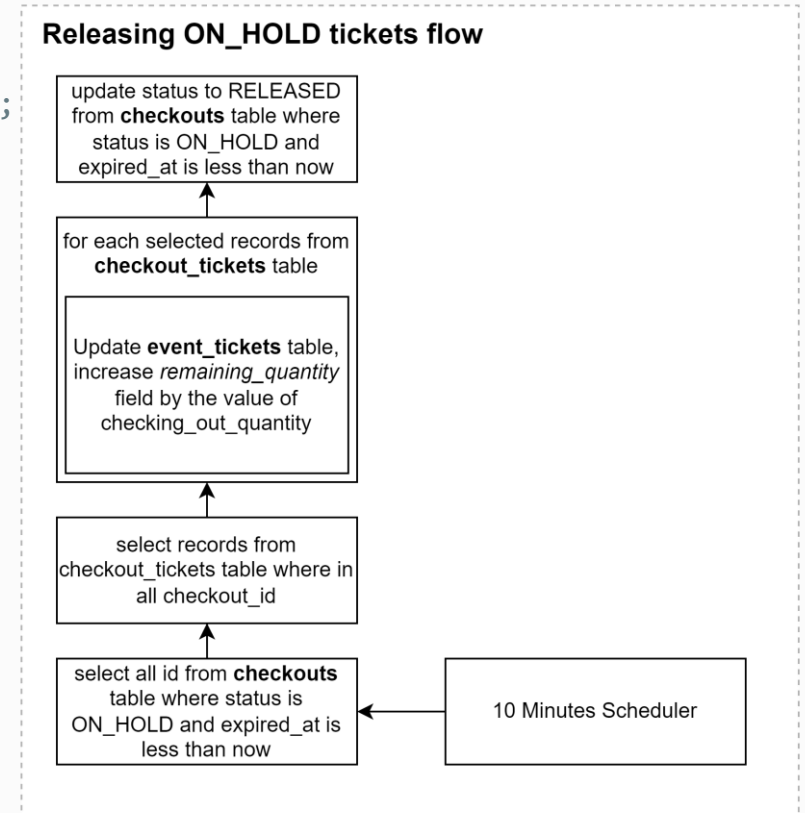
// Triggered every 10 minutes
async function releaseExpiredTickets() {
  // 1. Get IDs of expired 'ON_HOLD' checkouts
  const expiredIds = db.checkouts.find({ status: 'ON_HOLD', expired_at: { $lt: now() } });

  if (expiredIds.length === 0) return;
  // 2. Process within a transaction for data integrity
  db.transaction(() => {
    // Find all tickets related to these checkouts
    const tickets = db.checkout_tickets.find({ checkout_id: { $in: expiredIds } });

    // Restore quantity for each ticket
    tickets.forEach(t => {
      db.event_tickets.update(
        { id: t.event_id },
        { $inc: { remaining_quantity: t.checking_out_quantity } }
      );
    });

    // 3. Mark checkouts as RELEASED
    db.checkouts.updateMany(
      { id: { $in: expiredIds } },
      { $set: { status: 'RELEASED' } }
    );
  });
}

```



Algorithm or Pseudo Code

Send 

[Cookie](#)

 Schema [Beautify](#)

```
1
2     "identifier": "username@gmail.com", //user can login using username or email or phone_number
3     "password": "123456abcde"
4
```



API Contracts

Database & Performance Optimization

Relational Database Data modeling with denormalization

Database Schema Basics

- Master Tables
 - Core Business Objects
- Transaction Tables
 - Record Actions of Core Business Objects
 - Master table does something, a transaction record is created
- Pivot Tables
 - Junction, Mapping, or Cross-Reference
 - Build Many to many relations

Performance Optimization

Denormalizing a transaction table—essentially flattening the data so that a single row contains everything needed to understand the event—solves a very specific, very common bottleneck.

- Lightning-Fast Reads
- Historical Accuracy (Master Data Snapshot)
- Simpler Indexing

Denormalization

events	
id	bigint
title	varchar(255)
organizer_id	bigint
organizer_name	varchar(255)
event_template	event_templates E
type_id	bigint
type_name	varchar(255)
category_id	bigint
category_name	varchar(255)
slug	varchar(255)
alternative_domain	varchar(255)
tags	json
about	longtext
location_type	location_types E
venue_address	text
online_address	text
map_url	text
start_date	date
end_date	date
start_time	time
end_time	time
banner_images	json
agenda	json
faqs	json
additional_information	longtext
event_images	json
sponsors	json
contact_infomation	json

event_tickets	
id	bigint
name	varchar(255)
entry_date	date
price	decimal
currency	varchar(3)
event_id	int
ticket_type	ticket_types E
initial_quantity	int
remaining_quantity	int
sale_start_at	timestamp
sale_end_at	timestamp
description	text
early_price	decimal
early_start_at	timestamp
early_end_at	timestamp
ticket_visibility	ticket_visibilities E
created_at	timestamp
updated_at	timestamp
created_by	bigint
updated_by	bigint

customer_favorite_events	
id	bigint
customer_id	bigint
customer_name	varchar(255)
event_id	bigint

purchase_tickets	
id	bigint
purchase_id	bigint
customer_id	bigint
customer_name	varchar(255)
event_id	bigint
event_title	varchar(255)
event_ticket_id	bigint
event_ticket_name	varchar(255)
price	decimal(10,2)
currency	varchar(3)
platform_fee	decimal(10,2)
payment_fee	decimal(10,2)
tax_amount	decimal(10,2)
net_amount	decimal(10,2)
payout_amount	decimal(10,2)
ticket_number	varchar
attendee_name	varchar
attendee_email	varchar
attendee_phone	varchar
is_checked_in	bool
checked_in_at	timestamp
check_in_method	varchar(255)
check_in_location	varchar(255)
check_in_notes	text
created_at	timestamp
updated_at	timestamp
created_by	bigint
updated_by	bigint

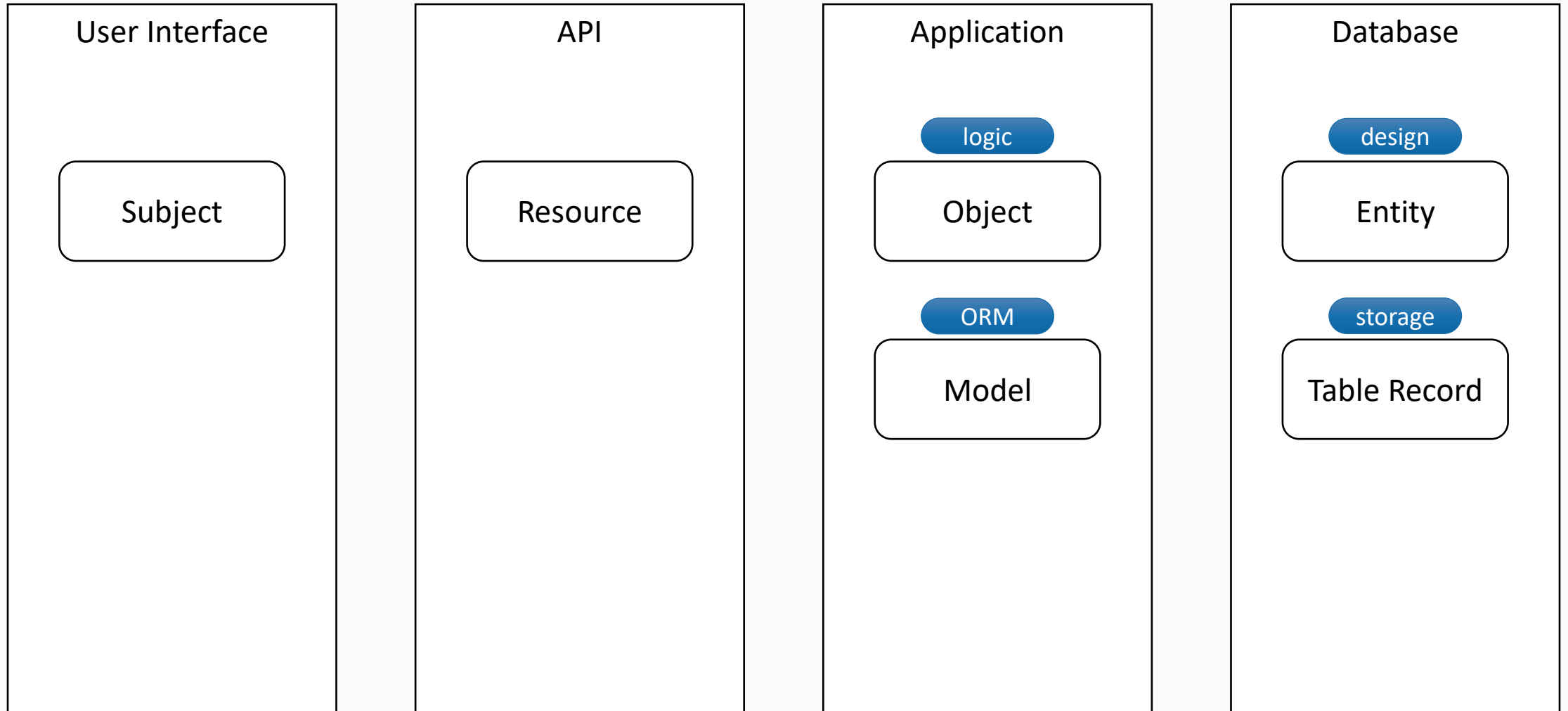
Building Scalable APIs

API Resource Management with ICRUD Pattern

Terms

- Subject – Resource or Object or Entity
- Action – Operations that users can perform on Subjects
 - Index, Create, Read, Update, Delete
- HTTP Request
 - Request Line
 - Method – GET, POST, PUT, PATCH, DELETE
 - URI – /index.php
 - Version – HTTP/1.1, HTTP/2
 - Headers
 - Payload

Subject



Index Action

Retrieve a list of resources implemented with filtering, sorting, and pagination to optimize the retrieval of large data sets.

Example Use Case:

An admin might retrieve the details of a specific user.

- Action: Read
- Subject: User

HTTP Request:

GET /users/123

Create Action

Add a new Subject to the system.

Example Use Case:

An admin might create a new User account.

- Action: Create
- Subject: User

HTTP Request:

POST /users

The request body would contain details such as name, email, and role for the new user.

Read Action

Retrieve a specific Subject by its unique identifier.

Example Use Case:

An admin might create a new User account.

- Action: Read
- Subject: User

HTTP Request:

GET /users/123

This request would retrieve the details of the User with ID 123, such as their name, email, and account status.

Update Action

Modify the data of an existing Subject.

Example Use Case:

An admin might update the role or email address of a User or modify their profile details.

- Action: Update
- Subject: User

HTTP Request:

PUT /users/123

The request body would contain the new details for the User with ID 123, such as a new email address or role.

Delete Action

Remove a Subject from the system.

Example Use Case:

An admin might delete a User from the system.

- Action: Delete
- Subject: User

HTTP Request:

`DELETE /users/123`

This request would delete the User with ID 123 from the system.

Extending Actions and Subjects

- The Action and Subject concepts described above—Index, Create, Read, Update, and Delete—are fundamental operations and work well in many common API scenarios. However, in more complex systems, you may encounter situations that go beyond these basic actions.
- Send Email, Activate User, Deactivate User, Assign Role, Generate Report, Approve Order, ...

Security & Error Handling

Authentication, Authorization and Rate Limiting

Custom Exception Class

API Authentication

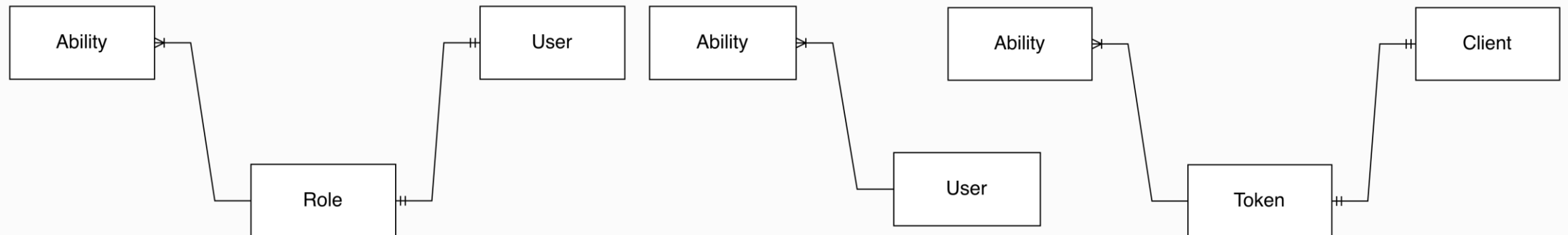
- Session-Based
- Token-Based
 - Stateful: **Opaque Token**
 - Stateless: **JWT**, PASETO, Macaroon, Biscuit, Branca, ...
- API Keys
- Basic HTTP
 - Authorization: Basic base64(username:password)
- OAuth 2.0
- mTLS
- ..., There are lots of mechanisms and you can invent one too 😊

API Authorization

- RBAC (Role-Based Access Control)
- ABAC (Attribute-Based Access Control)
- OAuth 2.0 Scopes (Delegated Authorization)
- ACLs (Access Control Lists)
- ..., There are lots of mechanisms and you can invent one too 😁

CASL – My Favorite

- <https://casl.js.org>
- Can be used in both Frontend and Backend for JavaScript
- **Ability**: The combination of Action and Subject that defines what a user is allowed to do.
- Can use the concept to build both RBAC and ABAC mechanisms
- You can build yourself in your backend if you are not using JavaScript
- I built myself for Laravel Framework, you can check it out here in [mojura starter kit](#)



Example Abilities

Let's imagine a system with Admin, Editor, and Viewer roles. Each role has a different set of abilities related to User management.

- Admin Abilities

- Read User: View user details.
- Create User: Add new users.
- Update User: Modify user information.
- Delete User: Remove users from the system.

- Editor Abilities

- Viewer Abilities

API Rate Limiting

1. Global (System-Wide) Limits
2. Tiered (Context-Aware) Limits
3. Endpoint (Resource-Specific) Limits
4. Security (Behavioral) Limits

Global (System-Wide) Limits

- Scope: The entire API/Gateway.
- Purpose: The "Circuit Breaker." It protects your infrastructure (CPU/RAM/Database) from total collapse if a massive, unexpected spike occurs.
- Example: "Limit all traffic to 10,000 requests per second (RPS) to prevent server crash."

Tiered (Context-Aware) Limits

- Scope: Based on the user's role or subscription status.
- Purpose: Monetization and Service Level Agreement (SLA) enforcement.
- Example: "Free tier users get 100 requests/minute, while Enterprise/Gold tier users get 5,000 requests/minute."

Endpoint (Resource-Specific) Limits

- Scope: Specific API routes.
- Purpose: Protecting "expensive" operations. Even a legitimate user shouldn't be able to run a heavy report generator 100 times per minute.
- Example:
 - GET /health -> 60 requests/minute.
 - POST /generate-heavy-invoice-pdf -> 2 requests/minute.

Security (Behavioral) Limits

- Scope: Identity/IP/Token.
- Purpose: Stopping abuse and brute-force attempts regardless of user tier.
- Example: "Maximum 5 failed login attempts per minute" or "50 requests/minute for password reset tokens."

Rate Limiting Summary

Layer	Type	Primary Goal
Global	Infrastructure	System Stability
Tiered	Business	Monetization/SLA
Endpoint	Resource	Cost/Compute Control
Security	Identity	Abuse Prevention

Production Deployment Practices

Think big, start small and scale as you grow

1-2-3 Go

- Phase 1: The "All-in-One"
 - Simplicity, speed to market, and low cost.
 - A single server (VPS or bare metal) handles everything
 - **Best For:** MVPs, internal tools, and low-traffic applications
- Phase 2: The "Separated"
 - Resource isolation and targeted performance tuning.
 - Application Server, Database Server, File Storage Server
 - Load Balancing Servers
 - **Best For:** Applications seeing consistent growth and needing higher uptime.
- Phase 3: The "Distributed"
 - High availability, fault tolerance, and independent scalability.
 - Service-oriented or microservices architecture. Components scale automatically based on demand
 - Databases might be sharded or clustered for read/write splitting
 - **Best For:** High-traffic platforms, enterprise-level SaaS, and complex applications requiring 99.99% uptime.



Thank You!

[Aung Kyaw Minn](#)

Solution Architect @ AYA Innovation Labs

